Holo-encoder 使用指南

本文补充原始 README，介绍环境准备、预测、网络生成与训练步骤。原始实现采用 MATLAB R2021a。

第一步：下载代码与模型

下载或克隆官方仓库。

可以使用发布的模型运行预测，也可以通过以下脚本构建并训练自己的模型。

文件或数据	获取方式	对应脚本
HoloEncoder_untrained.mat	下载，或运行网络生成脚本	HoloEncoder_generation.m
HoloEncoder_trained.mat	下载预训练模型，或自行训练并保存	HoloEncoder_training.m
训练与验证图像	另行下载 DIV2K，或准备自己的图像。仓库不生成数据集。	HoloEncoder_training.m 读取并预处理图像
相位全息图与模拟重建结果	加载训练好的模型，对目标图像运行预测	HoloEncoder_predict.m

将模型文件放在仓库根目录：

```
Holo-encoder/
+-- functions/
+-- images/
|   +-- 0805.png
+-- HoloEncoder_trained.mat
+-- HoloEncoder_untrained.mat       # 仅从头训练时需要
+-- HoloEncoder_generation.m
+-- HoloEncoder_predict.m
+-- HoloEncoder_training.m
+-- initializePlots.m
+-- reconstruction_comparison.m
+-- unet.m
+-- README.md
```

保留 functions/ 中的自定义层文件，加载模型前将该文件夹加入 MATLAB 路径。

第二步：准备 MATLAB 环境

参考环境为 MATLAB R2021a。需要以下产品：

- MATLAB
- Deep Learning Toolbox
- Image Processing Toolbox
- Parallel Computing Toolbox（原版 GPU 预测脚本需要）

建议使用 MATLAB 支持的 NVIDIA GPU。4K 分辨率的预测和训练需要较多内存与计算资源。

打开 MATLAB，将当前文件夹切换到仓库根目录。检查环境：

```
ver
canUseGPU
```

查看可用 GPU 的信息：

```
gpuDevice
```

原版预测脚本直接使用 gpuArray。如需尝试 CPU 执行，将输入转换语句中的 gpuArray 包装去掉。本文尚未验证 CPU 执行。

第三步：运行预测

使用 HoloEncoder_predict.m。这一步无需训练数据集。

脚本加载预训练模型和示例图像：

```
addpath('./functions');
load HoloEncoder_trained.mat
X = imread('./images/0805.png');
```

测试自己的图像时，修改 imread 中的路径。首次测试保留原版预处理：

```
X = im2gray(X);
X = imresize(X,[2160,3840]);
X = single(X);
```

运行：

```
HoloEncoder_predict
```

脚本通过以下语句准备 GPU 输入：

```
dlX = gpuArray(dlarray(X,'SSCB'));
```

SSCB 表示两个空间维、通道维和批次维。示例使用单通道灰度图像。保留 single(X)：它只改变数据类型，不会归一化图像数值。

第四步：查看并保存结果

预测脚本执行两次前向计算：

```
dlY = forward(dlnet,dlX,'Outputs','tanh');
dlZ = forward(dlnet,dlX);
```

- dlY 是相位全息图。名为 tanh 的层执行 pi*tanh(X)，输出约为 −π 至 π 弧度。
- dlZ 是物理传播模块计算得到的模拟重建光强。

脚本打开 hologram 和 reconstruction 两个图窗。相位图可能呈复杂纹理，应通过重建图判断目标图像是否恢复。

脚本已将相位和重建结果分别提取为 Y 和 Z。如需保存，在末尾添加：

```
save('HoloEncoder_result.mat','Y','Z');
```

第五步：生成未训练模型或选择已有权重

训练使用 HoloEncoder_training.m。首先选择网络初始化方式。

继续训练预训练模型

保留默认加载语句：

```
load HoloEncoder_trained.mat
```

训练从已有权重开始，但会重新初始化 Adam 优化器的统计量，因此不是完整恢复训练检查点。

从头训练

未训练模型可下载，也可通过 HoloEncoder_generation.m 生成。

将 MATLAB 当前文件夹设为仓库根目录，运行：

```
HoloEncoder_generation
```

脚本依次完成：

1. 设置输入尺寸与传播计算尺寸。
2. 调用 unet.m 构建相位编码网络。
3. 添加批归一化层与相位约束层。
4. 设置光学参数，连接 Fresnel 传播层。
5. 创建并保存网络：

```
dlnet = dlnetwork(lgraph);
save('HoloEncoder_untrained.mat','dlnet');
```

初始化后的网络以变量 dlnet 保存到 HoloEncoder_untrained.mat。这一步不读取训练图像，也不进行训练。

接着，将 HoloEncoder_training.m 中的加载语句改为：

```
load HoloEncoder_untrained.mat
```

完整流程为：

```
HoloEncoder_generation.m
    -> HoloEncoder_untrained.mat
    -> HoloEncoder_training.m（加载未训练模型）
    -> 训练后的模型文件
    -> HoloEncoder_predict.m（加载训练后的模型）
```

生成网络需要 unet.m 和 functions/，且会覆盖同名的 HoloEncoder_untrained.mat。

只使用发布的预训练模型进行预测时，无需生成网络。

第六步：设置数据路径并开始训练

原版训练脚本使用 DIV2K 图像。准备相互独立的训练与验证文件夹，例如：

```
Holo-encoder/
+-- DIV2K_train_HR/
+-- DIV2K_valid_HR/
|   +-- 0801.png
+-- ...
```

在 HoloEncoder_training.m 中设置路径：

```
trainImagePath = fullfile(pwd,'DIV2K_train_HR');
validImagePath = fullfile(pwd,'DIV2K_valid_HR');
```

同时修改训练脚本末尾预测部分的测试图像路径：

```
X = imread(fullfile(validImagePath,'0801.png'));
```

检查模型加载语句、数据路径和输出文件名，然后运行：

```
HoloEncoder_training
```

训练比较模拟重建图与目标图像，并更新网络。不需要配对的参考全息图。

训练参数包括 numEpochs、miniBatchSize 和 learnRate。显存不足时，先减小 miniBatchSize。原版循环对每个小批次连续更新 10 次。

默认保存为 HoloEncoder_trained.mat。为保留发布的模型，训练前可将保存语句改为：

```
save('HoloEncoder_trained_custom.mat','dlnet');
```

使用此模型预测时，同步修改 HoloEncoder_predict.m 中的模型加载文件名。

第七步：修改光学参数前检查配置

HoloEncoder_generation.m 的默认参数如下：

参数	默认值	含义
inputSize	[2160 3840]	输入图像与相位尺寸
outputSize	[3840 3840]	补零后的传播计算尺寸
lambda	532e-6	波长 532 nm，以 mm 表示
z	160	传播距离 160 mm
dp	0.00374	像素间距 3.74 μm

光学参数、相位尺寸、FFT 补零、重建采样及目标图像的缩放与裁剪必须保持匹配。

这些参数在网络构建时写入传播模块。修改生成脚本不会改变已经保存的 MAT 模型。适配其他光学系统时，应构建对应的网络，并评估是否需要重新训练或微调。

主要文件

文件	用途
HoloEncoder_predict.m	加载训练好的模型，生成相位与重建图
HoloEncoder_generation.m	构建并保存未训练网络
HoloEncoder_training.m	读取图像、计算损失、更新网络并保存模型
unet.m	构建带残差与跳跃连接的相位编码网络
initializePlots.m	创建训练曲线与图像显示窗口
reconstruction_comparison.m	对提供的 GS、Wirtinger 和 Holo-encoder 相位图进行角谱重建比较

functions/ 包含以下自定义层：

文件	用途
tanhpiLayer.m	通过 pi*tanh 约束输出相位
fresnelLayer.m	从相位计算 Fresnel 传播光强
plusLayer.m	添加预设项，在传播中用于二次相位
cosLayer.m、sinLayer.m	计算相位的余弦与正弦分量
ZeroPadding2dLayer.m	空间补零
fftshiftLayer.m	数组坐标移位
fft2DLayer.m	计算二维 FFT，输出实部与虚部
subtractionLayer.m	在传播计算中执行分量相减
intensityLayer.m	根据复场分量计算光强
hadamardProdLayer.m	逐元素乘法，是否使用取决于网络连接

主要计算流程为：

```
目标图像 -> unet -> tanhpiLayer -> fresnelLayer -> 重建光强
```

训练通过重建误差更新相位编码网络。训练后可以直接处理新图像，无需对每张图像重新训练。

常见问题

问题	检查方法
找不到模型或图像	将当前文件夹设为仓库根目录，检查路径
无法识别自定义层类	加载前运行 `addpath('./functions')`，保留所有依赖的层文件
GPU 执行错误	检查工具箱、GPU 支持情况与 MATLAB 环境
GPU 显存不足	减小训练批次大小；改变分辨率还需调整网络、传播和图像对齐
新版 MATLAB 加载或执行失败	以 R2021a 为参考，根据具体错误检查自定义层接口
训练损失为 NaN 或 Inf	检查输入和中间结果；常量目标可能使 NPCC 分母为零，可考虑加入小量 epsilon
彩色输入产生灰度结果	原版会将输入转为灰度；彩色全息需要按波长处理，并匹配显示系统
模拟正常但 SLM 结果不佳	检查相位标定、照明、波长、传播距离、像素间距与光路对准

`imshow(...,[])` 会对每幅图像单独拉伸显示范围。用于 SLM 时，应按器件标定转换相位值，不能将显示图直接视为标定后的相位图。

引用与链接

使用代码或模型时，请引用：

Jiachen Wu, Kexuan Liu, Xiaomeng Sui, and Liangcai Cao, “High-speed computer-generated holography using an autoencoder-based deep neural network,” Optics Letters 46, 2908-2911 (2021).
论文 DOI

- 官方仓库
- 模型下载
- 相关工作：4K-DMDNet
- 相关工作：WGCNet

最新联系方式请参见官方 README。

Holo-encoder User Guide

This guide supplements the original README with steps for setup, prediction, network generation, and training. The original implementation was developed for MATLAB R2021a.

Step 1: Download the code and model files

Download or clone the official repository.

Use the released model for prediction, or build and train your own model with the scripts below.

File or data	How to obtain it	Related script
HoloEncoder_untrained.mat	Download it, or generate it with the network generation script	HoloEncoder_generation.m
HoloEncoder_trained.mat	Download the pretrained model, or train and save your own model	HoloEncoder_training.m
Training and validation images	Download DIV2K separately, or prepare your own images. The repository does not generate the dataset.	HoloEncoder_training.m reads and preprocesses the images
Phase hologram and simulated reconstruction	Load a trained model and run prediction on a target image	HoloEncoder_predict.m

Place the model files in the repository root:

```
Holo-encoder/
+-- functions/
+-- images/
|   +-- 0805.png
+-- HoloEncoder_trained.mat
+-- HoloEncoder_untrained.mat       # Needed only for training from scratch
+-- HoloEncoder_generation.m
+-- HoloEncoder_predict.m
+-- HoloEncoder_training.m
+-- initializePlots.m
+-- reconstruction_comparison.m
+-- unet.m
+-- README.md
```

Keep the custom layer files in functions/. Add this folder to the MATLAB path before loading a model.

Step 2: Set up MATLAB

The reference environment is **MATLAB R2021a**. Required products are:

- MATLAB

- Deep Learning Toolbox

- Image Processing Toolbox

- Parallel Computing Toolbox for the original GPU prediction script

A supported NVIDIA GPU is recommended. Prediction and training at 4K resolution require substantial memory and computation.

Open MATLAB and set the current folder to the repository root. Check your environment:

```
ver
canUseGPU
```

To inspect an available GPU, run:

```
gpuDevice
```

The original prediction script uses gpuArray directly. To try CPU execution, remove the gpuArray wrapper from the input conversion line. CPU execution has not been verified in this guide.

Step 3: Run prediction

Use **HoloEncoder_predict.m**. No training dataset is needed for this step.

The script loads the pretrained model and a sample image:

```
addpath('./functions');
load HoloEncoder_trained.mat
X = imread('./images/0805.png');
```

To use your own image, change the path in imread. Keep the original preprocessing for your first test:

```
X = im2gray(X);
X = imresize(X,[2160,3840]);
X = single(X);
```

Run:

```
HoloEncoder_predict
```

The script prepares the GPU input with:

```
dlX = gpuArray(dlarray(X,'SSCB'));
```

SSCB denotes two spatial dimensions, the channel dimension, and the batch dimension. The example uses a single grayscale channel. Keep single(X) as written: it changes the data type without normalizing the image values.

Step 4: View and save the results

The prediction script performs two forward passes:

```
dlY = forward(dlnet,dlX,'Outputs','tanh');
dlZ = forward(dlnet,dlX);
```

- dlY is the phase hologram. The layer named tanh applies pi*tanh(X), giving phase values approximately between -π and π radians.

- dlZ is the simulated reconstruction intensity computed by the physical propagation module.

The script opens two figures: hologram and reconstruction. A phase hologram may look like a complex texture. Use the reconstruction to check whether the target image is recovered.

The script already extracts the phase as Y and the reconstruction as Z. To save them, add this line at the end:

```
save('HoloEncoder_result.mat','Y','Z');
```

Step 5: Generate an untrained model or choose existing weights

Use **HoloEncoder_training.m** for training. First choose how to initialize the network.

Continue training a pretrained model

Keep the default loading statement:

```
load HoloEncoder_trained.mat
```

Training starts from the learned weights with newly initialized Adam optimizer statistics; this is not a full training checkpoint resume.

Train from scratch

An untrained model can be downloaded or generated with **HoloEncoder_generation.m**.

With MATLAB's current folder set to the repository root, run:

```
HoloEncoder_generation
```

The script:

1. Sets the input and propagation calculation sizes.
2. Calls unet.m to build the phase encoder.
3. Adds batch normalization and the phase constraint layer.
4. Sets the optical parameters and connects the Fresnel propagation layer.
5. Creates the network and saves it:

```
dlnet = dlnetwork(lgraph);
save('HoloEncoder_untrained.mat','dlnet');
```

This saves the initialized network as dlnet in HoloEncoder_untrained.mat. It does not read training images or perform training.

Next, change the loading statement in HoloEncoder_training.m to:

```
load HoloEncoder_untrained.mat
```

The full workflow is:

```
HoloEncoder_generation.m
    -> HoloEncoder_untrained.mat
    -> HoloEncoder_training.m (load the untrained model)
    -> trained model file
    -> HoloEncoder_predict.m (load the trained model)
```

Network generation requires unet.m and functions/ and overwrites an existing HoloEncoder_untrained.mat.

You do not need to generate a network when using the released pretrained model for prediction.

Step 6: Set the data paths and start training

The original training script uses DIV2K images. Prepare separate training and validation folders, for example:

```
Holo-encoder/
+-- DIV2K_train_HR/
+-- DIV2K_valid_HR/
|   +-- 0801.png
+-- ...
```

In HoloEncoder_training.m, set the paths:

```
trainImagePath = fullfile(pwd,'DIV2K_train_HR');
validImagePath = fullfile(pwd,'DIV2K_valid_HR');
```

Also update the test image path in the prediction section at the end of the training script:

```
X = imread(fullfile(validImagePath,'0801.png'));
```

Check the model loading statement, data paths, and output filename. Then run:

```
HoloEncoder_training
```

Training compares the simulated reconstruction with the target image and updates the network. **Paired reference holograms are not required.**

Training settings include numEpochs, miniBatchSize, and learnRate. If GPU memory is insufficient, reduce miniBatchSize. The original loop performs 10 updates on each mini-batch.

By default, training saves HoloEncoder_trained.mat. To preserve the released model, change the save statement before training:

```
save('HoloEncoder_trained_custom.mat','dlnet');
```

When using this model for prediction, update the loading statement in HoloEncoder_predict.m to match the new filename.

Step 7: Check the configuration before changing optical parameters

The defaults in HoloEncoder_generation.m are:

Parameter	Default	Meaning
inputSize	[2160 3840]	Input image and phase size
outputSize	[3840 3840]	Padded propagation calculation size
lambda	532e-6	Wavelength of 532 nm, expressed in mm
z	160	Propagation distance of 160 mm
dp	0.00374	Pixel pitch of 3.74 μm

The optical parameters, phase size, FFT padding, reconstruction sampling, and target resizing and cropping must remain consistent.

These parameters are stored in the propagation module when the network is built. Editing the generation script does not change a previously saved MAT model. For a different optical setup, build the corresponding network and assess whether retraining or fine-tuning is needed.

Main files

File	Purpose
HoloEncoder_predict.m	Load a trained model and generate the phase and reconstruction
HoloEncoder_generation.m	Build and save an untrained network
HoloEncoder_training.m	Read images, compute the loss, update the network, and save the model
unet.m	Build the phase encoder with residual and skip connections
initializePlots.m	Create training plots and image displays
reconstruction_comparison.m	Compare angular-spectrum reconstructions of the supplied GS, Wirtinger, and Holo-encoder phase images

The functions/ folder contains the custom layers:

File	Purpose
tanhpiLayer.m	Constrain the output phase with pi*tanh
fresnelLayer.m	Compute Fresnel propagation from phase to intensity
plusLayer.m	Add a preset term, used for the quadratic phase in propagation
cosLayer.m, sinLayer.m	Compute cosine and sine components of the phase
ZeroPadding2dLayer.m	Apply spatial zero padding
fftshiftLayer.m	Shift array coordinates
fft2DLayer.m	Compute the 2D FFT and output its real and imaginary parts
subtractionLayer.m	Subtract components in the propagation calculation
intensityLayer.m	Compute intensity from complex field components
hadamardProdLayer.m	Perform element-wise multiplication; use depends on network connections

To understand the main workflow, follow:

```
Target image -> unet -> tanhpiLayer -> fresnelLayer -> reconstruction intensity
```

Training updates the phase encoder using the reconstruction error. After training, new images can be processed without retraining the network for each image.

Common issues

Issue	What to check
Model or image not found	Set the current folder to the repository root and check the paths
Custom layer class not recognized	Run addpath('./functions') before loading the model and keep all required layer files
GPU execution error	Check the toolboxes, GPU support, and MATLAB environment
Insufficient GPU memory	Reduce the training batch size. Changing resolution also requires changes to the network, propagation, and image alignment.
Loading or execution fails in a newer MATLAB release	Use R2021a as the reference environment and check the custom layer interfaces against the specific error
Training loss becomes NaN or Inf	Check inputs and intermediate values. Constant targets can cause a zero NPCC denominator; a small epsilon may help.
A color input produces a grayscale result	The original script converts inputs to grayscale. Color holography needs wavelength-specific processing and a suitable display setup.
Simulation works but the SLM result is poor	Check phase calibration, illumination, wavelength, propagation distance, pixel pitch, and optical alignment

The display command imshow(...,[]) rescales each image for viewing. For SLM use, convert phase values using the device calibration rather than treating the displayed image as a calibrated phase map.

Citation and links

If you use this code or its models, please cite:

Jiachen Wu, Kexuan Liu, Xiaomeng Sui, and Liangcai Cao, “High-speed computer-generated holography using an autoencoder-based deep neural network,” Optics Letters 46, 2908-2911 (2021). https://doi.org/10.1364/OL.425485

- Official repository
- Model download
- Related work: 4K-DMDNet
- Related work: WGCNet

For current contact details, see the official README.